

Payment Settings by Order Type

A store can now accept different payment methods for delivery than for collection — cash on pickup but not at the door, EC-Karte only in-store. The backend is live-ready; nothing uses it yet.

Backend committed `aa1d531` Branch `testServer` Breaking changes none

IF YOU CHANGE NOTHING	WHAT'S NEW	ROLLOUT
Your client keeps working, unchanged, forever.	One extra field on a response you already read.	Per client, on your own schedule. No coordinated release.

01 What actually changed

One endpoint you already call grew one new field.

`GET /store-payment-settings/{store_id}` returns everything it returned before — `cash_enabled`, `stripe_enabled`, `paypal_client_id` and the rest, all with the same meanings. It now *also* returns `by_order_type`:

```
{
  "store_id": 42,
  // — unchanged. still the store-wide default. —
  "cash_enabled": true,
  "card_enabled": true,
  "stripe_enabled": true,
  "paypal_enabled": false,
  "ec_enabled": true,
  "paypal_client_id": "AXk9...",

  // — new. already resolved — just read your block. —
  "by_order_type": {
    "delivery": { "cash_enabled": false, "card_enabled": true,
                  "stripe_enabled": true, "paypal_enabled": false,
                  "ec_enabled": false },
    "collection": { "cash_enabled": true, "card_enabled": true,
                    "stripe_enabled": true, "paypal_enabled": false,
                    "ec_enabled": true },
    "dine_in": { "cash_enabled": true, "card_enabled": true,
                 "stripe_enabled": true, "paypal_enabled": false,
                 "ec_enabled": true }
  }
}
```

All three blocks are always present and always fully populated, whether or not the store has configured that order type. There is no `null`, no missing key, and no fallback for you to

implement — the backend has already applied the store default to anything the owner left alone.

Today every store returns three identical blocks, because no store has configured anything yet. That's the point: you can ship against this now and see no behaviour change until owners start using it.

02 Who needs to do what

Two of the four clients have real work. The other two may have none — that's a question for you, not an assumption I've made.

CLIENT	WORK	WHAT IT IS
Restaurant ordering sites	Required	Checkout must offer payment methods from the customer's chosen order type, not the flat fields. See §3.
Owner app	Required	Payment settings screen needs a per-order-type editor. See §4.
Windows EXE	Confirm	Only if it displays or selects payment methods. If it just prints what the order already carries, nothing to do.
CRM	Confirm	Only if staff create orders or edit a store's payment config there.
Marketplace (B2C)	Later	Same checkout rule as the ordering sites when it reaches checkout.

03 Checkout

The customer-facing change. Small, with one trap in it.

Where you currently read `settings.cash_enabled` to decide whether to show the Cash button, read the block for the order type the customer has selected:

```
// before
const methods = settings;

// after — orderType is whatever the customer picked
const key = orderType === DELIVERY ? "delivery"
           : orderType === COLLECTION ? "collection"
           : "dine_in";
const methods = settings.by_order_type[key];

if (methods.cash_enabled) show(CASH);
if (methods.ec_enabled) show(EC_KARTE);
if (methods.stripe_enabled) show(CARD_ONLINE);
```

One `GET` returns all three blocks, so switching order type needs **no second request** — just re-read from the response you already hold.

THE TRAP — PLEASE DON'T SKIP THIS

A customer can pick a payment method *and then change their order type*. They choose Collection, select **Cash**, then switch to Delivery — where the store has cash turned off.

If you only filter the buttons on render, the stale selection survives and the order posts with a method the store doesn't accept for that type. The backend will not stop it (see §6), so the restaurant gets an order it can't take payment for.

On every order-type change, re-validate the current selection against the new block and clear it if it is no longer allowed. Make the customer pick again rather than silently substituting one for them.

Worth handling too: if a store disables every method for one order type, that order type should be unselectable at checkout rather than leading to a dead end with no payment buttons.

04 The owner settings screen

Where the restaurant actually configures this. Three endpoints.

GET /store-payment-settings/{store_id}/order-types

Populates the screen. Returns all three order types, resolved, each with an `is_override` flag.

PUT /store-payment-settings/{store_id}/order-types/{order_type}

Saves one order type. Partial — send only what changed.

DELETE /store-payment-settings/{store_id}/order-types/{order_type}

Resets that order type back to the store-wide default. Idempotent.

What comes back

```
[
  { "store_id": 42, "order_type": 1, "order_type_name": "delivery",
    "is_override": true,
    "flags": { "cash_enabled": false, "card_enabled": true,
              "stripe_enabled": true, "paypal_enabled": false,
              "ec_enabled": false } },
  { "store_id": 42, "order_type": 2, "order_type_name": "collection",
    "is_override": false, "flags": { ... } },
  { "store_id": 42, "order_type": 3, "order_type_name": "dine_in",
    "is_override": false, "flags": { ... } }
]
```

`flags` is always what's actually in effect, so bind your toggles straight to it. `is_override` is presentation only — it tells you whether this order type has its own settings (`true`) or is

following the store-wide default (`false`), so you can label it "Using store default" and offer a **Reset** that calls `DELETE` .

Saving

```
PUT /store-payment-settings/42/order-types/delivery
{ "cash_enabled": false, "ec_enabled": false }
```

Omitted methods keep their current value. On the first save for an order type the backend seeds all five from the store-wide default and then applies what you sent, so a partial save never silently switches something off. Sending all five every time is equally fine and probably simpler if your toggles submit as a set.

Auth is unchanged: admin, store owner, and multi-store owner can write; anyone can read. A non-permitted role gets `403` .

Suggested shape

Three tabs — Delivery, Collection, Dine-in — each with the same five toggles, a "Using store default" note when `is_override` is false, and a Reset. The existing store-wide screen stays as it is; it's now the default that unconfigured order types inherit, which is worth saying on screen in one line.

05 Reference

Order types

The `{order_type}` path segment accepts the name, the integer, or the alias — all three resolve to the same record, so use whichever fits your client.

NAME	INT	ALIAS	BACKEND CONSTANT
delivery	1	—	ORDER_TYPE_DELIVERY
collection	2	pickup	ORDER_TYPE_PICKUP
dine_in	3	—	ORDER_TYPE_DINE_IN

Anything else returns `400` with the allowed list in the message. Note the naming split: the customer-facing word is **collection**, the backend constant has always been `PICKUP` , and both work in the URL.

The five methods

FIELD	IS
cash_enabled	Cash, paid on handover

FIELD	IS
card_enabled	Card, generic
stripe_enabled	Card online via Stripe, paid up front
paypal_enabled	PayPal, paid up front
ec_enabled	EC-Karte, paid offline at the terminal

`paypal_client_id` is deliberately *not* per-order-type — PayPal credentials belong to the store. Keep reading it from the top level of the response, exactly as now.

06 Two things the backend does not do

It does not reject a disabled method. Order creation is untouched. If a client posts a payment method the store has switched off for that order type, the order is accepted. The flags decide what you *offer*; they are not a server-side guard. This is why the re-validation in §3 matters — it's the only thing standing between a stale selection and an unpayable order. We can add enforcement later; say the word if you'd rather have it before you ship.

It does not resolve anything on your side. Don't reimplement the "override, else store default" logic in the client. Both `by_order_type` and `flags` arrive already resolved. A second implementation in each frontend is four places to drift.

07 How to test it

Against a test store, once the backend is deployed.

- 1 Load checkout before touching anything. Every order type offers the same methods as today — confirms nothing regressed.
- 2 `PUT .../order-types/delivery` with `{"cash_enabled": false}`. Cash disappears from delivery checkout, stays on collection.
- 3 Select Cash on Collection, then switch to Delivery. **The selection must clear.** This is the one that breaks.
- 4 Change a store-wide toggle. Delivery keeps its own value; collection and dine-in follow the change.
- 5 `DELETE .../order-types/delivery`. Delivery goes back to matching the store default and the screen shows "Using store default".
- 6 Point an *old* client build at the same store. It must behave exactly as before — that's the guarantee the staggered rollout rests on.

08 What I need back from you

- Does the Windows EXE display or select payment methods, or only print what the order already carries?
- Does the CRM let staff create orders or edit a store's payment config?
- Which client owns the owner-facing settings screen — the app, the CRM, or both?
- Do you want server-side enforcement before you ship, or is offer-side filtering enough?
- Rough dates per client, so we know when the first restaurant can actually be switched on.

Backend: `app/routes/store_payment_routes.py`, `app/models/models.py`,
`app/models/schemas.py`

New table `store_payment_type_settings` – created automatically on server restart, no migration to run.

Verified with 35 assertions plus HTTP round-trips before commit.